SOFTWARE REQUIREMENTS SPECIFICATION

MiniOS — Memory and Task Management Subsystems

VERSION 3.2 — FINAL SUBMISSION

Group 31 | IIT Jodhpur | B.Tech CSE 2024

Date: April 15, 2026

Name	Roll No.	Sprint Ownership	Domain
Aman Yadav	B24CS1006	S1, S2	Memory Management
Raunak Kumar	B24CS1062	S3, S4	Interrupts & Scheduler
Palthyavath Jalendhar	B24CS1051	S5	Integration & NFR Validation
Shardul Diwate	B24CS1028	S0, S6	Build, Docs & Submission

REVISION HISTORY

Version	Date	Author(s)	Description of Changes
1.0	21 Jan 2026	Group 31	Initial draft submitted
2.0	04 Feb 2026	Group 31	Added sprint logic and technical refinements
3.0	25 Feb 2026	Group 31	Structural corrections, UML alignment, coded interfaces & NFRs
3.1	25 Feb 2026	Group 31	Added Section 7: Project Organization and Engineering Practices per professor feedback
3.1.2	10 Apr 2026	Group 31	Minor editorial corrections; sprint status updated to reflect S3–S4 completion
3.2	15 Apr 2026	Group 31	**Final submission. All sprints S0–S6 complete. Post-integration bug register appended (R-16–R-19). NFR evidence table added. 14/14 NFR tests PASS.**

v3.1.2 → v3.2 Change Summary

Section	What Changed in v3.2
Section 7.2 (Sprint Status)	All sprint statuses updated to COMPLETED. S5 integration results documented (14/14 tests PASS).
Section 7.2 (Sprint Details)	Each sprint now includes: what was built, key implementation decisions, failures encountered, and fixes applied.
Section 7.3 (Risk Register)	Four post-SRS discovered risks added: R-16 (.resume ret missing), R-17 (heap bitmap overlap), R-18 (registers_t layout inversion), R-19 (sched_add_task STI race). All resolved.
Section 7.4 (Project Management)	Trello replaced with Claude-based AI progress management tool. Sprint cards and task tracking methodology updated.
Appendix B (UML Inventory)	Diagram inventory confirmed against final implementation. Delivery status column added.
New: Appendix C (NFR Evidence)	Formal evidence table mapping every NFR to its measured test result from the S5 suite (14 PASS, 0 FAIL).

New: Appendix D (Deviation Log)	SRS v3.1 spec vs actual implementation deviations formally documented with justification.
New: Appendix E (Bug Register)	Complete bug log: 6 bugs found and fixed across S2–S5, each with root cause and fix applied.

TRACEABILITY MATRIX (v1.0 → v3.2)

All requirements have been implemented and tested. Final status is shown for each item.

Req ID	Component	v1.0 → v2.0	v3.0/v3.1 Change	v3.2 Evidence	Status
REQ-MEM-01 REQ-MEM-02	Memory Management (GDT + Paging)	Basic GDT/paging definitions added	Formal CR0/CR3 sequence, 2-level address decomposition specified	arch/x86/gdt.c, paging.c — fully implemented ✓	**Completed**
REQ-MEM-03 REQ-MEM-04	Physical Allocator + Kernel Heap	Bitmap allocator and 4MB heap introduced	mm_alloc_frame()/mm_free_frame() interfaces specified; heap above 0xC0000000	mm/mm_phys.c, mm/heap.c — heap bitmap reservation bug fixed (R-17) ✓	**Completed**
REQ-MEM-05	Kernel-User Isolation	Kernel above 0xC0000000 defined	Supervisor bit enforcement via PTE flags mandated	NFR-SEC-01: 0 violations across 2048+ kernel PTEs ✓	**Completed**
REQ-TASK-01 REQ-TASK-02	IDT + Timer Interrupt	256-entry IDT and IRQ0 at 100Hz specified	PIT port addresses (0x43/0x40), EOI signal, DPL=0 Interrupt Gate formally required	arch/x86/idt.c, pic.c, pit.c — 100Hz confirmed by S5 PERF-02 ✓	**Completed**
REQ-TASK-03 REQ-TASK-04	Scheduler + Context Switch	Round-Robin + register save/restore described	Full register set (EAX–EBP, EIP, ESP, EFLAGS, seg regs), IRET sequence, circular queue formally specified	arch/x86/context_switch.asm, scheduler/sched.c — .resume ret bug fixed (R-16) ✓	**Completed**
REQ-TASK-05	Task Control Block	TCB referenced implicitly in scheduler	Formal TCB structure defined: register state, kernel stack ptr, page dir ref, state enum, PID	include/task.h, scheduler/task.c — offsets verified via _Static_assert ✓	**Completed**

1. Introduction

1.1 Purpose

This document constitutes the Software Requirements Specification (SRS) for MiniOS, a 32-bit x86 educational operating system kernel developed as part of the course project at IIT Jodhpur. **Version 3.2 is the final submission document, reflecting the fully implemented and tested system.**

The document specifies, in precise and unambiguous terms, the functional and non-functional requirements governing the following two core subsystems:

- Memory Management Subsystem (Sprints S1, S2)
- Task (Process) Management Subsystem (Sprints S3, S4)

This SRS defines system behavior, hardware interaction constraints, external interface boundaries, and architectural assumptions. It serves as the authoritative reference for design validation, implementation traceability, and academic evaluation. Version 3.2 additionally documents all deviations from earlier specifications, bugs discovered and resolved during integration, and formal evidence of NFR compliance.

1.2 Document Conventions

All requirements are identified using a structured coding scheme to ensure traceability:

Identifier Prefix	Category	Example
REQ-MEM-XX	Memory Management functional requirements	REQ-MEM-01
REQ-TASK-XX	Task Management functional requirements	REQ-TASK-03
EXT-UI-XX	User interface specifications	EXT-UI-01
EXT-HW-XX	Hardware interface specifications	EXT-HW-02
EXT-SW-XX	Software interface specifications	EXT-SW-01
EXT-COM-XX	Communication interface specifications	EXT-COM-01
NFR-PERF-XX	Performance non-functional requirements	NFR-PERF-01
NFR-SEC-XX	Security non-functional requirements	NFR-SEC-01
NFR-REL-XX	Reliability non-functional requirements	NFR-REL-01
NFR-MAINT-XX	Maintainability non-functional requirements	NFR-MAINT-01

NFR-PORT-XX	Portability non-functional requirements	NFR-PORT-01

Hexadecimal values follow standard x86 notation (e.g., 0xC0000000). All memory sizes are expressed in bytes unless explicitly stated otherwise. Text marked ✓ denotes requirements fully implemented and verified in v3.2.

1.3 Intended Audience

This document is intended for the following audiences:

- Kernel Developers: Team members implementing the memory and task management subsystems.
- System Architects: Personnel responsible for validating structural and architectural design decisions.
- Academic Evaluators: Instructors assessing the completeness, rigor, and correctness of engineering deliverables.

Readers are expected to possess working knowledge of C programming, NASM assembly, and x86 protected-mode architecture.

1.4 Product Scope

MiniOS is a modular monolithic 32-bit kernel operating in x86 protected mode. It is designed as an educational platform to demonstrate low-level operating system concepts including memory management, virtual addressing, interrupt handling, and preemptive multitasking.

Scope Category	Component	Included?
Memory Management	Global Descriptor Table (GDT) initialization	**YES ✓**
Memory Management	Two-level paging (Page Directory + Page Tables)	**YES ✓**
Memory Management	Physical memory bitmap allocator	**YES ✓**
Memory Management	Kernel heap allocation (kmalloc / kfree)	**YES ✓**
Task Management	Interrupt Descriptor Table (IDT) setup	**YES ✓**
Task Management	Timer-based preemptive scheduling (PIT)	**YES ✓**
Task Management	Round-Robin task scheduler	**YES ✓**
Task Management	Context switching with full register preservation	**YES ✓**

Out of Scope	File system	**NO**
Out of Scope	Networking stack	**NO**
Out of Scope	Graphical User Interface (GUI)	**NO**
Out of Scope	Multi-core / SMP scheduling	**NO**
Out of Scope	Advanced IPC (pipes, sockets, shared memory)	**NO**

1.5 References

- Intel 64 and IA-32 Architectures Software Developer's Manual, Intel Corporation.
- Arpaci-Dusseau, R. and Arpaci-Dusseau, A., Operating Systems: Three Easy Pieces, Arpaci-Dusseau Books.
- OSDev Wiki — https://wiki.osdev.org
- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Multiboot Specification v0.6.96

2. Overall Description

2.1 Product Perspective

MiniOS is a self-contained 32-bit kernel that operates directly on x86 hardware, replacing the standard OS layer entirely. It does not depend on any existing operating system at runtime. The system is designed to be booted via a Multiboot-compliant bootloader (GRUB), after which it assumes full control of the hardware.

Hardware Component	Purpose in MiniOS
CPU Control Registers (CR0, CR3)	Enable paging and configure page directory base address
Memory Management Unit (MMU)	Virtual-to-physical address translation via two-level paging
Programmable Interrupt Controller (PIC)	Remap hardware IRQs to avoid conflict with CPU exception vectors
Programmable Interval Timer (PIT)	Generate periodic timer interrupts (100 Hz) for preemptive scheduling
VGA Text Buffer (0xB8000)	80×25 character display for boot messages and runtime output
COM1 Serial Port (0x3F8)	Out-of-band debug logging at 9600 baud, 8N1

2.2 Product Functions

All of the following core functions are **fully implemented** in v3.2:

- Physical memory tracking using a bitmap-based frame allocator
- Virtual address translation via two-level paging (Page Directory + Page Tables)
- Dynamic kernel heap memory allocation and deallocation (kmalloc / kfree)
- Hardware interrupt handling through a fully populated 256-entry IDT
- Timer-driven preemptive multitasking via PIT channel 0 and IRQ0
- Round-Robin process scheduling with circular task queue
- Safe and complete CPU context switching using IRET-based register restore

2.3 User Classes and Characteristics

The primary user class of MiniOS is the kernel developer — a team member who extends the system with new modules or subsystems. Such users must possess proficiency in C (freestanding), NASM assembly, x86 architecture, and low-level debugging tools (GDB with QEMU). There is no end-user

graphical interface; all interaction occurs at the kernel level via VGA text output and COM1 serial port logging.

2.4 Operating Environment

Parameter	Value
CPU Architecture	32-bit x86 (i386 / i686)
Operating Mode	Protected Mode (32-bit), entered by GRUB before kernel_main
Bootloader	Multiboot-compliant (GRUB 2); magic 0x1BADB002, flags 0x00000003
Emulation Environment	QEMU (qemu-system-i386); flags: -cpu qemu32 -smp 1 -m 64M
Cross-Compiler	i686-elf-gcc 13.2.0 (freestanding target); flags: -ffreestanding -fno-builtin -nostdlib -nostdinc
Assembler	NASM (Netwide Assembler) 2.x
Build Host	Ubuntu 22.04 LTS
Kernel Load Address	0x00100000 (1 MB physical)
Kernel Virtual Base	0xC0000000 (higher-half kernel)

2.5 Design and Implementation Constraints

- Language Constraint: The kernel is implemented exclusively in freestanding C and NASM Assembly. No standard C library (glibc or similar) is used.
- Memory Alignment: All page structures (Page Directory and Page Tables) must be 4 KB aligned.
- Paging Granularity: 4 KB pages exclusively. No large page (4 MB) or PAE support.
- Physical Memory Limit: The allocator manages only RAM reported by the Multiboot memory map.
- Kernel Load Address: Linker script (linker.ld) loads kernel at physical address 0x100000, with 4 KB-aligned sections and kernel_start / kernel_end symbols.
- Build Toolchain: i686-elf-gcc cross-compiler with -ffreestanding -fno-builtin -nostdlib -nostdinc. Host compilers are not permitted.
- Hardware Register Isolation: All CR0/CR3 writes, LGDT/LIDT instructions, and port I/O are confined to arch/x86/ per NFR-PORT-01.

2.6 Assumptions and Dependencies

- Target hardware (QEMU qemu-system-i386) supports an MMU capable of x86 two-level paging.
- PIT and PIC are available and respond to standard x86 I/O port commands.

- Multiboot-compliant bootloader provides a valid memory map structure to the kernel at boot.
- Cross-compilation toolchain (i686-elf-gcc 13.2.0) is properly installed and configured on the development machine.
- Team members possess the prerequisite knowledge of x86 architecture and low-level C required to work on their assigned subsystems.

3. External Interface Requirements

3.1 User Interface

EXT-UI-01 VGA Text Console: The system shall provide an 80×25 VGA text mode console, accessible at physical address 0xB8000, for kernel diagnostic output, boot messages, and runtime logs. The console shall support colored character output using VGA attribute bytes.

Source File	kernel/vga.c
Functions	vga_init(), vga_puts(), vga_putchar(), vga_set_color(), vga_clear(). Scrolling supported.
Status	**IMPLEMENTED ✓**

3.2 Hardware Interfaces

EXT-HW-01 MMU Control via CR0 and CR3: The kernel shall configure virtual memory by writing the page directory physical base address into the CR3 register, and enable paging by setting bit 31 (PG bit) of the CR0 register. These operations constitute the memory management hardware interface.

Source File	arch/x86/paging_asm.asm
Functions	paging_load_directory() — writes physical PD address to CR3; paging_enable() — sets CR0.PG bit 31.
Status	**IMPLEMENTED ✓**

EXT-HW-02 PIC Remapping: The system shall remap the Programmable Interrupt Controller (PIC) IRQ lines to interrupt vectors 0x20–0x2F by programming the PIC through I/O ports 0x20 and 0xA0. This prevents conflict with CPU exception vectors 0x00–0x1F.

Source File	arch/x86/pic.c
Functions	pic_remap() — Master PIC → 0x20, Slave PIC → 0x28; pic_send_eoi(irq) — writes 0x20 to port 0x20 (and 0xA0 for IRQs 8–15).
Status	**IMPLEMENTED ✓**

EXT-HW-03 PIT Configuration: The Programmable Interval Timer (PIT) shall be configured via I/O port 0x43 (Mode/Command Register) and 0x40 (Channel 0 data port) to generate periodic IRQ0 interrupts at a frequency of 100 Hz, producing a timer tick every 10 milliseconds.

Source File	arch/x86/pit.c
Functions	pit_init() — outb(0x43, 0x36); divisor 11932 → port 0x40 lo/hi; pit_irq0_handler(); pit_get_ticks().
Actual Frequency	99.997 Hz (divisor 11932 gives 1,193,182/11932 Hz; error < 0.003%)
Status	**IMPLEMENTED & VERIFIED ✓ — 1000 ticks in exactly 10 seconds (NFR-PERF-02)**

3.3 Software Interfaces

EXT-SW-01 Memory–Scheduler Interface: The Memory Management subsystem shall expose page directory allocation and deallocation interfaces (paging_create_directory(), paging_free_directory()) to the Task Management subsystem. The scheduler shall use these interfaces when creating or destroying process address spaces.

Source File	arch/x86/paging.c; tested in kernel/s5_integration.c
Test Function	s5_test_ext_sw01() — creates PD, verifies zeroed, maps user page, frees PD. 4 sub-tests PASS.
Note	All kernel tasks share a single CR3 (kernel page directory). paging_create/free_directory() interface verified standalone. See Appendix D for deviation justification.
Status	**IMPLEMENTED & TESTED ✓**

3.4 Communication Interfaces

EXT-COM-01 Serial Port Debug Logging: Debug and diagnostic messages shall be transmitted through the COM1 serial port (I/O base address 0x3F8) at a baud rate of 9600, 8N1.

Source File	kernel/serial.c
Functions	serial_init(), serial_puts(), serial_putchar(), serial_printf() (%s %d %u %x %c), serial_puts_hex(). FIFO enabled.
Status	**IMPLEMENTED ✓ — all kernel diagnostic output routed through COM1**

4. System Features

4.1 Memory Management Subsystem

The Memory Management subsystem is responsible for all aspects of physical and virtual memory resource management in MiniOS. It provides the foundational infrastructure upon which task isolation and virtual addressing are built. **Priority: High — all requirements COMPLETED in Sprints S1 and S2.**

4.1.1 Functional Requirements — REQ-MEM-01 to REQ-MEM-05

REQ-MEM-01 Segmentation — GDT Initialization	
Source Files	arch/x86/gdt.c, arch/x86/gdt_flush.asm
Functions	gdt_init(), gdt_set_entry(), gdt_flush()
Segment 0	Null descriptor — mandatory x86 requirement
Segment 1	Kernel code — selector 0x08, access 0x9A, granularity 0xCF (DPL=0, 32-bit, 4KB pages, base=0, limit=4GB)
Segment 2	Kernel data — selector 0x10, access 0x92, granularity 0xCF (DPL=0, 32-bit, 4KB pages, base=0, limit=4GB)
Far Jump	gdt_flush() in NASM: lgdt [gdt_ptr] → jmp 0x08:.flush → reloads CS; DS/ES/FS/GS/SS loaded with 0x10
Evidence	Serial log: [S1] GDT initialized
Status	**IMPLEMENTED ✓**

REQ-MEM-02 Paging — Two-Level Address Translation	
Source Files	arch/x86/paging.c, arch/x86/paging_asm.asm
Key Functions	paging_create_directory(), paging_map_page(), paging_identity_map_first4mb(), paging_map_kernel(), paging_load_directory(), paging_enable()
Address Decomposition	Bits 31–22: Page Directory index | Bits 21–12: Page Table index | Bits 11–0: Page offset (4 KB)
Identity Map	virt 0x00000000–0x003FFFFF → phys 0x00000000–0x003FFFFF — required so CPU can fetch the next instruction after CR0.PG is set
Kernel Map	virt 0xC0000000–0xC03FFFFF → phys 0x00100000–0x004FFFFF (Supervisor bit set, PAGE_RW)

Heap Map	virt 0xC0400000–0xC07FFFFF → phys 0x00500000–0x008FFFFF (Supervisor bit set, PAGE_RW)
Enable Sequence	1. Create PD 2. Identity map 3. Map kernel 4. Map heap 5. CR3 ← PD physical addr 6. CR0.PG ← 1
Evidence	Serial log: [KERNEL] Paging ENABLED
Status	**IMPLEMENTED ✓**

REQ-MEM-03 Physical Memory Allocation — Bitmap Allocator	
Source File	mm/mm_phys.c
Functions	mm_init(), mm_alloc_frame(), mm_free_frame(), mm_test()
Bitmap Size	1M bits (32,768 uint32_t words) covering 4 GB at 4 KB granularity; 1 = allocated, 0 = free
Initialization Order	1. Mark ALL frames allocated 2. Parse Multiboot map, free AVAILABLE regions above 1 MB 3. Re-protect kernel frames (0x100000–kernel_end) 4. Reserve heap backing (0x500000–0x8FFFFF) [Bug fix R-17]
Heap Reservation	Frames 0x500000–0x8FFFFF explicitly marked allocated in mm_init() to prevent mm_alloc_frame() from handing out heap backing frames. This was a post-integration bug fix (R-17, B-01).
Evidence	Serial log: [mm_init] ready. Kernel: 0x00100000 – 0x0012D000
Status	**IMPLEMENTED ✓**

REQ-MEM-04 Kernel Heap Allocation	
Source File	mm/heap.c
Functions	mm_heap_init(), kmalloc(), kfree(), mm_heap_dump()
Allocator Design	Implicit free list, first-fit, with block splitting and forward coalescing on kfree()
Block Layout	block_header_t (12 bytes: size, used, _pad[3], *next) | usable data | ...
Virtual Range	0xC0400000–0xC07FFFFF (4 MB, ~4095 KB usable after header)
Physical Backing	0x00500000–0x008FFFFF (reserved in bitmap, explicitly not allocated by mm_alloc_frame())
Minimum Allocation	16 bytes (prevents tiny slivers that cannot be reused)
Thread Safety	CLI/STI guards wrap the entire free-list walk in both kmalloc() and kfree() — added during S5 integration (R-13, B-05)

Evidence	Serial log: [HEAP] Initialised + S5 heap stress test: 4 phases PASS
Status	**IMPLEMENTED ✓**

REQ-MEM-05 Kernel–User Memory Isolation	
Enforcement	PAGE_USER flag deliberately omitted in paging_map_kernel() and paging_map_heap(). All kernel PTEs have U/S bit (bit 2) cleared.
Violation Trigger	Any user-mode access to a kernel page triggers Page Fault (ISR 14), which logs CR2 + error code and halts.
Test Function	s5_test_sec01_kernel_isolation() — reads CR3, walks PD entries 768–1023, checks U/S bit = 0 on every present PTE
Result	U/S violations found: 0 across 2048+ kernel PTEs checked
Evidence	Serial log: [PASS] NFR-SEC-01: all kernel PTEs have U/S=0
Status	**IMPLEMENTED & VERIFIED ✓**

4.2 Task Management Subsystem

The Task Management subsystem provides preemptive multitasking by implementing interrupt handling, a timer-driven scheduler, and hardware-level context switching. It depends on the Memory Management subsystem. **Priority: High — all requirements COMPLETED in Sprints S3 and S4.**

4.2.1 Functional Requirements — REQ-TASK-01 to REQ-TASK-05

REQ-TASK-01 IDT Initialization	
Source Files	arch/x86/idt.c, arch/x86/idt_flush.asm, arch/x86/isr_stubs.asm
Functions	idt_init(), idt_set_gate(), idt_flush()
Entry Format	8-byte 32-bit Interrupt Gate: handler low 16b | selector 0x08 | flags 0x8E (Present, DPL=0, 32-bit) | handler high 16b
Entries 0–31	CPU exception stubs — ISR_NOERRCODE / ISR_ERRCODE macros in isr_stubs.asm
Entries 32–47	Hardware IRQ stubs — IRQ_STUB macro (after PIC remap to vectors 0x20–0x2F)

Entries 48–255	Default-filled with Present=0 (flags=0x0E) — stray vectors trigger controlled #NP instead of jumping to address 0 [Bug fix B-02]
IDTR Load	idt_flush() in NASM executes lidt [idtr]
Evidence	Serial log: [S3] IDT loaded (256 gates)
Status	**IMPLEMENTED ✓**

REQ-TASK-02 Timer Interrupt Generation	
Source File	arch/x86/pit.c
Functions	pit_init(), pit_irq0_handler(), pit_get_ticks()
Configuration	outb(0x43, 0x36) — channel 0, lobyte/hibyte, square wave; divisor 11932 written to port 0x40 (lo then hi byte)
Actual Frequency	99.997 Hz — 1,193,182 Hz / 11,932 = 99.997 Hz (error < 0.003%)
EOI	pic_send_eoi(0) — writes 0x20 to port 0x20 — called in pit_irq0_handler() before sched_switch()
Call Chain	IRQ0 → common_irq_stub → irq_handler() → pit_irq0_handler() → pic_send_eoi() → sched_switch()
Evidence	Serial log: [PASS] NFR-PERF-02: PIT fires within 1% of 100 Hz; 1000 ticks in 10s
Status	**IMPLEMENTED & VERIFIED ✓**

REQ-TASK-03 Round-Robin Scheduler	
Source File	scheduler/sched.c
Functions	sched_init(), sched_add_task(), sched_next(), sched_switch(), sched_start()
Queue Structure	Circular singly-linked list; queue_tail->next = head (head is always queue_tail->next)
Quantum	10 ms (one PIT tick at 100 Hz)
Switch Logic	sched_switch(): old = current_task → new = sched_next() → old.state = READY → new.state = RUNNING → current_task = new → context_switch(old, new)
Guard	sched_switch() is a no-op when current_task == NULL (before sched_start()). This allows S5 tests to run safely with IRQs enabled.
Bootstrap	sched_start() uses a dummy boot_tcb as the 'old' task — context is saved but never restored; IRET jumps into Task A

Evidence	Serial log: [TASK A] PID=1 running → [TASK B] PID=2 running → [TASK C] PID=3 running → repeating
Status	**IMPLEMENTED ✓**

REQ-TASK-04 Context Switch — Full Register Preservation	
Source File	arch/x86/context_switch.asm
Function	void context_switch(tcb_t *old_task, tcb_t *new_task)
Save Sequence (outgoing)	PUSHFD (force IF=1 via OR) → push CS (0x08) → push .resume (EIP) → push DS → PUSHAD (8 GPRs, 32 bytes) → mov [old+8], esp
Restore Sequence (incoming)	mov esp, [new+8] → mov cr3, [new+12] → POPAD → restore DS/ES/FS/GS → IRET (pops EIP, CS, EFLAGS)
New Task IRET	IRET pops fn_ptr from initial frame → task function starts executing with IF=1
Existing Task IRET	IRET pops .resume → ret → returns through sched_switch → pit_irq0_handler → irq_handler → common_irq_stub's own IRET → task resumes
Critical Bug Fixed	.resume: ret was accidentally removed during S4 debugging. Without it, the CPU fell through to garbage bytes → #UD → double fault → triple fault → infinite QEMU reset loop. Restored in S5 integration (R-16, B-06).
Latency	~300 cycles @ ~1 GHz QEMU ≈ 0.3 μs — 167× under the 50 μs budget
Status	**IMPLEMENTED & VERIFIED ✓**

REQ-TASK-05 Task Control Block (TCB) Structure	
Source Files	include/task.h, scheduler/task.c
Functions	task_create(fn), task_destroy(tcb)
TCB Fields	pid (offset 0), state enum (offset 4: READY/RUNNING/BLOCKED/TERMINATED), kernel_esp (offset 8), page_dir physical address (offset 12), *next pointer (offset 16), *kernel_stack base (offset 20)
TCB Size	24 bytes (6 × uint32_t)
Offset Verification	_Static_assert in scheduler/task.c verifies kernel_esp=8 and page_dir=12 match TCB_KERNEL_ESP/TCB_PAGE_DIR constants in context_switch.asm at compile time
Initial Stack Frame	task_create() hand-crafts: [EFLAGS=0x0202, CS=0x08, EIP=fn_ptr, DS=0x10, then 8×0 for GPRs] — kernel_esp points to bottom of GPR block
Stack Size	8 KB per task — allocated via kmalloc()

Evidence	Serial log: [TASK] Created PID=N stack_base=0x... kernel_esp=0x...
Status	**IMPLEMENTED ✓**

5. Non-Functional Requirements

All NFRs have been formally verified in Sprint 5 via the automated test suite in kernel/s5_integration.c. Full evidence is provided in **Appendix C**. Final score: **14 PASS, 0 FAIL.**

5.1 Performance Requirements

NFR-PERF-01 Context Switch Latency < 50 µs: The time elapsed between the triggering of the timer interrupt (IRQ0) and the completion of the context switch shall not exceed 50 microseconds under normal operating conditions.

Test Function	s5_test_perf01_context_switch() in kernel/s5_integration.c
Method	RDTSC cycle counter (verified via CPUID) over 1000 sched_next() calls. PIT resolution (10ms) is too coarse for single-switch measurement — TSC provides nanosecond precision.
Measured Result	~250–1000 total cycles / 1000 iterations ≈ 0.25–1.0 µs average
Budget	50 µs = ~50,000 cycles at ~1 GHz (QEMU)
Result	**PASS ✓ — margin: ~167× under budget**

NFR-PERF-02 Timer Stability at 100 Hz: The PIT shall generate interrupts at exactly 100 Hz. The timer frequency shall remain stable across the duration of system operation, with no observable clock drift.

Test Function	s5_test_perf02_pit_stability() in kernel/s5_integration.c
Method	Count PIT ticks over 10-second busy-wait using pit_get_ticks(); tolerance ±1% (990–1010 ticks).
Measured Result	1000 ticks in exactly 10 seconds
Result	**PASS ✓ — exactly at target**

5.2 Security Requirements

NFR-SEC-01 Page-Level Access Control: Paging shall enforce memory access control through the User/Supervisor bit in every page table entry. Kernel pages shall set the Supervisor bit (U/S=0), preventing user-mode code from accessing kernel memory. Any attempt by a user-mode process to access a kernel page shall trigger a Page Fault exception (interrupt vector 14).

Test Function	s5_test_sec01_kernel_isolation() in kernel/s5_integration.c
Method	SIDT to find IDT; read CR3 to find active PD; walk PD indices 768–1023 (0xC0000000+ virtual range); verify U/S bit (bit 2) = 0 on every present PTE.
PTEs Checked	2048+ kernel PTEs across 4 MB kernel map + 4 MB heap map
Violations Found	0
Result	**PASS ✓ — zero violations**

5.3 Reliability Requirements

NFR-REL-01 Fault Handling: The kernel shall install dedicated ISRs for CPU exception vectors 13 (General Protection Fault) and 14 (Page Fault). Upon triggering, these ISRs shall log a descriptive fault message (including fault code and CR2 address for page faults) to the VGA console and serial port, then halt the kernel.

Source Files	arch/x86/isr_stubs.asm, arch/x86/isr_handler.c, arch/x86/isr14.c
ISR 13 (GPF)	Logs error code to serial + VGA, then executes HLT. Handler in arch/x86/isr_handler.c.
ISR 14 (PF)	Reads CR2 (faulting address) via paging_get_cr2(), logs CR2 + error code, then HLT. Handler in arch/x86/isr14.c.
Test Method	SIDT → read IDT[14] → reconstruct handler address from off_lo + off_hi → verify ≠ 0.
Result	**PASS ✓ — IDT[14] handler addr: 0x001035XX (non-zero)**

5.4 Maintainability Requirements

NFR-MAINT-01 Modular Directory Structure & Naming: All source code shall be organized into subsystem-specific directories. Mandatory structure: boot/, arch/x86/, mm/, scheduler/, include/. All functions shall use prescribed module prefixes: mm_, paging_, sched_, isr_, pit_, pic_.

Implemented Dirs	boot/, arch/x86/, mm/, scheduler/, kernel/, include/ — matching NFR-MAINT-01 structure plus kernel/ for non-arch runtime code

Prefix Compliance	mm_init, mm_alloc_frame, paging_map_page, paging_enable, sched_init, sched_switch, isr_handler, pit_init, pit_get_ticks, pic_remap, pic_send_eoi — all conform
Header Guards	All include/*.h files use #ifndef XX_H / #define XX_H / #endif pattern
SRS Traceability	Every function includes /* Implements: REQ-XX */ comment directly above the function definition
Result	**PASS ✓**

5.5 Portability Requirements

NFR-PORT-01 Architecture Isolation: All hardware-specific code (CR0/CR3 manipulation, LGDT/LIDT, I/O port access, NASM assembly routines) shall be isolated within arch/x86/. Code in mm/ and scheduler/ shall interact with hardware only through abstraction layers defined in arch/x86/.

CR0 write	arch/x86/paging_asm.asm — paging_enable() only
CR3 write	arch/x86/paging_asm.asm — paging_load_directory(); arch/x86/context_switch.asm — task switch
CR2 read	arch/x86/paging_asm.asm — paging_get_cr2()
outb/inb	arch/x86/pit.c, arch/x86/pic.c, arch/x86/gdt.c; kernel/serial.c (COM1 only, no control registers)
LGDT / LIDT	arch/x86/gdt_flush.asm, arch/x86/idt_flush.asm
Violation Check	No CR0/CR3/port I/O found in mm/, scheduler/, or include/ — verified statically in S5
Result	**PASS ✓**

6. Other Requirements

Cross-Compiler: The build system uses i686-elf-gcc 13.2.0 cross-compiler to produce freestanding x86 ELF binaries. Compiled with -ffreestanding -fno-builtin -nostdlib -nostdinc. Host compilers are not permitted. ✓ Verified — all team members built successfully.

Linker Script: Custom linker.ld places .multiboot section first, loads kernel at 0x100000, aligns .text/.rodata/.data/.bss sections to 4 KB boundaries, and provides kernel_start and kernel_end symbols used by mm_init() to locate the end of kernel code. ✓ Verified via readelf -S kernel.elf.

Multiboot Header: boot/boot.asm includes a valid Multiboot header in the .multiboot section: magic 0x1BADB002, flags 0x00000003, checksum = -(magic + flags). GRUB verifies the magic number and provides the memory map in the multiboot_info_t structure passed to kernel_main. ✓ Verified — serial log: Multiboot magic: 0x2BADB002.

7. Project Organization and Engineering Practices

This section documents the process-oriented requirements governing team structure, development methodology, tooling, and coding standards.

7.1 Team Structure and Alpha Manager Model

The team operates under an Alpha Manager model. An Alpha Manager is the designated technical owner of a subsystem, responsible for subsystem-level architectural decisions, code review of contributions to their module, integration validation, and approval of merge requests affecting their subsystem. This model ensures clear subsystem ownership, reduces architectural conflicts, accelerates debugging, and provides defined accountability during integration testing.

Subsystem	Alpha Manager	Core Responsibilities
Memory Management	Aman Yadav	Paging design, bitmap frame allocator validation, heap management, kernel-user isolation (Sprints S1, S2)
Task Management	Raunak Kumar	Scheduler logic, context switch implementation, IDT/PIT/PIC interrupt handling (Sprints S3, S4)
Integration & Validation	Palthyavath Jalendhar	NFR test suite design and execution, integration testing, bug discovery and resolution during S5
Build & Documentation	Shardul Diwate	Linker script, Makefile, cross-compiler setup, RTM, demo script, code freeze, submission (Sprints S0, S6)

Alpha Managers hold approval authority over merge requests touching their module. No code is merged into a subsystem without the Alpha Manager's review and sign-off.

7.2 Development Methodology — Sprint Plan

The methodology follows a bottom-up, dependency-driven incremental model. Each sprint delivers validated outputs before the next sprint begins. The dependency chain is:

S0 (Boot) → S1 (Physical Memory) → S2 (Virtual Memory + Heap) → S3 (Interrupts) → S4 (Scheduler) → S5 (Integration) → S6 (Documentation)

This strict ordering ensures each subsystem builds upon a validated lower layer, eliminating circular dependencies and minimising integration risk.

7.2.1 Sprint Status Summary

Sprint	Module	Owner	Status	Evidence
S0	Environment & Boot Infrastructure	Shardul Diwate	**COMPLETE** ✓	Boots to VGA + serial in QEMU; [BOOT] messages on COM1
S1	Segmentation & Physical Memory	Aman Yadav	**COMPLETE** ✓	Serial: [S1] GDT initialized, [mm_init] ready
S2	Virtual Memory & Heap	Aman Yadav	**COMPLETE** ✓	Serial: [KERNEL] Paging ENABLED, [HEAP] Initialised
S3	Interrupt Infrastructure	Raunak Kumar	**COMPLETE** ✓	Serial: [S3] IDT loaded, PIC remapped, PIT 100Hz
S4	Scheduler & Context Switch	Raunak Kumar	**COMPLETE** ✓	Serial: [TASK A/B/C] PID=N cycling round-robin
S5	Integration & NFR Validation	Palthyavath Jalendhar	**COMPLETE** ✓	14 PASS, 0 FAIL — all NFRs verified (see Appendix C)
S6	Documentation & Submission	Shardul Diwate	**COMPLETE** ✓	RTM, demo.md, risk log, SRS v3.2, ISO built and frozen

7.2.2 Sprint Details

Sprint 0 — Environment & Boot Infrastructure	
Alpha Manager	Shardul Diwate
Mapped Requirements	EXT-UI-01 (VGA), EXT-COM-01 (Serial), EXT-HW-01 (Multiboot), NFR-MAINT-01 (dir structure)
Objective	Establish stable low-level development environment and boot infrastructure before kernel implementation begins.
What Was Built	i686-elf-gcc 13.2.0 toolchain built from source. NASM and QEMU configured. Linker script (linker.ld) placing kernel at 0x100000 with 4KB-aligned sections and kernel_end symbol. boot/boot.asm with Multiboot header (magic 0x1BADB002, flags 0x00000003). 16KB initial stack. VGA driver (80×25, 0xB8000, colored output). COM1 serial logger (9600 baud, 8N1, FIFO). Makefile with targets: all, iso, run, debug, clean, freeze. Directory structure per NFR-MAINT-01.

Issues / Resolutions	No open issues. Cross-compiler version incompatibilities across team members resolved by pinning i686-elf-gcc 13.2.0 and documenting build steps in demo.md. QEMU flags pinned to -cpu qemu32 -smp 1 -m 64M for consistency (R-02).
Deliverable	Kernel boots to VGA console and serial in QEMU. Protected mode entered by GRUB. [BOOT] Serial logger initialized + [BOOT] VGA driver initialized on COM1.
Status	**COMPLETED ✓**

Sprint 1 — Segmentation & Physical Memory	
Alpha Manager	Aman Yadav
Mapped Requirements	REQ-MEM-01 (GDT), REQ-MEM-03 (Physical Allocator)
Objective	Establish foundational memory segmentation and physical memory tracking infrastructure.
What Was Built	GDT with 3 descriptors in flat model: null (index 0), kernel code (selector 0x08, DPL=0, 32-bit, limit=4GB), kernel data (selector 0x10, DPL=0). gdt_flush() in NASM performs lgdt then far jump jmp 0x08:.flush to reload CS; DS/ES/FS/GS/SS loaded with 0x10. Physical frame bitmap allocator: 1 bit per 4KB frame (1M bits total). mm_init() parses Multiboot memory map, pre-marks all frames below kernel_end as allocated, then frees AVAILABLE regions above 1MB. mm_alloc_frame() returns physical address of first free frame. mm_free_frame(addr) validates 4KB alignment and clears bit.
Issues / Resolutions	GDT flush far jump verified via GDB segment register inspection (R-03/R-04). Bitmap boundary checks added to prevent overwriting kernel memory (R-05). No blocking issues — completed on time.
Testing	Allocated and freed 100 frames; bitmap integrity checked via GDB. Segment registers confirmed correct after gdt_flush().
Deliverable	Stable segmentation confirmed. Functional physical memory allocator. Serial: [S1] GDT initialized + [mm_init] ready.
Status	**COMPLETED ✓**

Sprint 2 — Virtual Memory & Kernel Heap	
Alpha Manager	Aman Yadav
Mapped Requirements	REQ-MEM-02 (Paging), REQ-MEM-04 (Heap), REQ-MEM-05 (Kernel-User Isolation)

Objective	Enable paging, enforce kernel–user memory isolation, and provide dynamic heap allocation.
What Was Built	Two-level page table setup in the mandatory sequence: (1) create 4KB-aligned PD, (2) identity-map first 4MB so CPU can fetch instructions after CR0.PG is set, (3) map kernel at 0xC0000000 with Supervisor bit, (4) map heap at 0xC0400000, (5) load CR3, (6) set CR0.PG. Page fault handler (ISR 14) reads CR2 and logs faulting address + error code. Kernel heap: 4MB implicit free-list allocator with block splitting and forward coalescing. Minimum allocation 16 bytes.
Issues / Resolutions	Paging enable before identity map caused immediate triple fault (R-06) — resolved by ensuring identity map step precedes CR0.PG. Heap CLI/STI guards deferred to S5 (heap was single-threaded in S2 isolation). Post-integration discovery (R-17, B-01): heap physical frames 0x500000–0x8FFFFF were not reserved in the bitmap, causing mm_alloc_frame() to hand them out and overwrite live heap data under multi-task load. Fixed in mm_init() by explicitly marking those frames allocated after the kernel protection loop.
Testing	Intentional page fault at 0xDEADBEEF tested. User access violation verified. Heap stress: alloc/free 50 blocks, pattern-fill and verify, fragmentation test.
Deliverable	Fully functional virtual memory. Kernel executes at 0xC0000000+. Serial: [KERNEL] Paging ENABLED + [HEAP] Initialised.
Status	**COMPLETED ✓**

Sprint 3 — Interrupt Infrastructure	
Alpha Manager	Raunak Kumar
Mapped Requirements	REQ-TASK-01 (IDT), REQ-TASK-02 (PIC remap + PIT at 100Hz)
Objective	Establish hardware interrupt capability as the foundation for preemptive scheduling.
What Was Built	256-entry IDT with 32-bit interrupt gates (Present, DPL=0, selector 0x08). ISR stubs generated by NASM macros: ISR_NOERRCODE (pushes dummy 0 + vector), ISR_ERRCODE (CPU already pushed error code), IRQ_STUB (IRQ stubs). common_isr_stub and common_irq_stub save full task state: PUSHAD + segment registers. 8259A PIC remapped: Master → 0x20, Slave → 0x28 (to avoid conflict with CPU exception vectors 0–31). PIT configured at 100Hz via divisor 11932 (outb to 0x43 then 0x40). GPF (ISR 13) and PF (ISR 14) fault handlers installed.
Issues / Resolutions	IDT default fill changed from flags=0x8E (Present=1, base=0x00000000) to flags=0x0E (Present=0) for entries 48–255 (B-02). Original Present=1 with null handler caused stray interrupts on those vectors to jump to address 0 and triple fault. Present=0 causes a controlled #NP exception instead. No other blocking issues.

Testing	Divide-by-zero (ISR 0) tested. Timer tick counter verified incrementing at 10ms intervals in GDB. IRQ0 handler confirmed via serial output. GPF and PF handlers confirmed active.
Deliverable	Stable interrupt-driven kernel. Serial: [S3] IDT loaded + [S3] PIC remapped + [S3] PIT configured at 100Hz.
Status	**COMPLETED ✓**

Sprint 4 — Scheduler, TCB & Context Switch	
Alpha Manager	Raunak Kumar
Mapped Requirements	REQ-TASK-03 (Round-Robin Scheduler), REQ-TASK-04 (Context Switch), REQ-TASK-05 (TCB)
Objective	Implement preemptive multitasking with safe and complete CPU context switching.
What Was Built	TCB struct (24 bytes, 6 fields: pid, state, kernel_esp, page_dir, *next, *kernel_stack). Offsets verified by _Static_assert. task_create(fn) allocates TCB + 8KB kernel stack via kmalloc() and hand-crafts the initial IRET frame (EFLAGS=0x0202 for IF=1, CS=0x08, EIP=fn_ptr, DS=0x10, 8×0 GPRs). context_switch() in NASM: PUSHFD (force IF=1) + push CS + push .resume + push DS + PUSHAD = 48-byte save frame → save ESP to old->kernel_esp → load new ESP → load CR3 from new->page_dir → POPAD + restore segment regs + IRET. Round-robin circular queue with sched_start() using dummy boot_tcb.
Issues / Resolutions	Latent bug discovered in S5 integration (R-16, B-06): the ret instruction after the .resume label in context_switch.asm was accidentally removed during a debugging session that confused the ESP state after IRET with the ESP state before the save block. Without ret, the CPU fell through to garbage bytes after .resume, triggering #UD → double fault → triple fault → infinite QEMU reset loop. The bug was not visible in S4 standalone tests (only two tasks, no full integration). Fixed in S5 by restoring ret. Additionally, sched_add_task() STI race identified (R-19): bare CLI+STI would re-enable interrupts even if the caller had them disabled, allowing IRQ0 before task_count was incremented. Fixed by replacing with EFLAGS pushf+cli/push+popf.
Testing	Tasks A, B, C in isolation: verified perfect round-robin A→B→C→A output. Register preservation verified via GDB snapshot comparison across context switches.
Deliverable	Preemptive multitasking kernel. Serial: [TASK A] PID=1 running → [TASK B] PID=2 running → [TASK C] PID=3 running (repeating).
Status	**COMPLETED ✓**

Sprint 5 — Integration & NFR Validation

Alpha Manager	Palthyavath Jalendhar
Mapped Requirements	EXT-SW-01, NFR-PERF-01/02, NFR-SEC-01, NFR-REL-01, NFR-MAINT-01, NFR-PORT-01
Objective	Integrate all subsystems, run the complete automated NFR test suite, and resolve any integration issues discovered.
What Was Built	kernel/s5_integration.c with s5_run_all_tests() executing 8 test functions before the scheduler starts. Tests: s5_test_ext_sw01() (create/verify/free PD), s5_test_perf01_context_switch() (RDTSC over 1000 sched_next() calls), s5_test_perf02_pit_stability() (1000 ticks in 10 seconds), s5_test_sec01_kernel_isolation() (PTE U/S bit walk), NFR-REL-01 test (IDT[14] handler address check), s5_test_heap_stress() (30 allocs, pattern write, fragment, re-alloc, free all), s5_audit_naming_and_arch() (static audit of prefix compliance and arch isolation). Tests run with STI enabled (IRQs on) but current_task=NULL so sched_switch() is a no-op during the entire test sequence.
Key Integration Failure	After all 14 S5 tests passed, the kernel entered an infinite reboot loop: Tasks A/B/C ran briefly, then QEMU reset and repeated. Root cause (R-16, B-06): .resume: ret was absent in context_switch.asm. After IRET to .resume, the CPU fell through to garbage bytes → #UD → double fault → triple fault → machine reset. Investigation required disassembling context_switch.o with objdump and checking the QEMU debug log for the triple fault sequence. Fix: restoring ret at .resume. Lesson: unit testing in isolation (S4) cannot catch bugs that only manifest under full system integration.
Other Bugs Fixed	B-01 (R-17): Heap physical frames not reserved in bitmap — fixed in mm_init(). B-03 (R-18): registers_t field order inverted vs NASM PUSHAD — corrected in include/isr.h. B-04 (R-19): sched_add_task() unconditional STI — replaced with EFLAGS save/restore. B-05 (R-13): kmalloc/kfree not interrupt-safe — CLI/STI guards added to both.
Final Test Result	**14 PASS, 0 FAIL — STATUS: ALL TESTS PASSED**
Status	**COMPLETED ✓**

Sprint 6 — Documentation & Submission	
Alpha Manager	Shardul Diwate
Objective	Final validation, deliverable packaging, and submission preparation.
Deliverables Produced	RTM.md — Requirement Traceability Matrix mapping all 10 FRs + 6 NFRs + 4 EXTs to source file, function, and test evidence. demo.md — build steps, full annotated expected serial output, demo script, Q&A reference. Project_review_log.md — risk register R-01 to R-19 (15 original + 4 post-integration), sprint-by-sprint review notes. SRS v3.2 (this document). Bootable MiniOS.iso built via grub-mkrescue. Code freeze via git tag v1.0-final.
Status	**COMPLETED ✓**

7.3 Comprehensive Risk Mitigation Plan

Low-level operating system development involves direct hardware interaction, assembly-level debugging, privilege management, and tight subsystem interdependencies. A single kernel fault may crash the entire system (triple fault). The following risk register documents all identified risks, their mitigations, and final resolution status.

7.3.1 Original Risk Register (R-01 to R-15) — All Resolved

ID	Risk Description	Prob.	Impact	Mitigation Strategy	Status
R-01	Cross-compiler not available on team members' machines	High	High	Documented install steps in demo.md; pinned i686-elf-gcc 13.2.0; Raunak hosted shared binary during setup	**RESOLVED**
R-02	QEMU version incompatibility causing x86 emulation differences	Med	Med	Pinned QEMU flags: -cpu qemu32 -smp 1 -m 64M; tested on Ubuntu 22.04 LTS	**RESOLVED**
R-03	Multiboot header misalignment (magic not in first 8KB)	Low	High	Linker script places .multiboot section first; verified with readelf -S kernel.elf	**RESOLVED**
R-04	GDT flush far jump corrupting segment registers	Med	High	gdt_flush() in NASM; far jump to CS=0x08 flushes segment cache atomically; verified in GDB	**RESOLVED**
R-05	Physical allocator bitmap overwriting kernel memory	Med	High	mm_init() marks all frames below kernel_end as allocated before scanning Multiboot map	**RESOLVED**
R-06	Paging enable before identity map causes immediate page fault	High	Critical	Identity-map first 4MB before enabling CR0.PG; load CR3 before CR0 set; sequence enforced in kernel_main	**RESOLVED**
R-07	Heap overlapping kernel BSS/data or bitmap handing out heap frames	Med	Critical	Heap starts at 0xC0400000 (above 4MB kernel map). BUG FIX (R-17): heap physical frames 0x500000–0x8FFFFF reserved in mm_init() bitmap	**RESOLVED**
R-08	IDT handler stubs not saving all registers; registers_t struct layout wrong	High	High	common_isr_stub saves PUSHAD + segment regs. BUG FIX (R-18): registers_t field order corrected to match NASM PUSHAD order	**RESOLVED**
R-09	PIC spurious IRQs before remap causing exceptions	Med	Med	pic_remap() called before STI; master→0x20, slave→0x28 established before any IRQ fires	**RESOLVED**

R-10	PIT divisor rounding error causing drift > 1%	Low	Med	Divisor 11932 → 99.997 Hz (0.003% error); verified S5 PERF-02: 1000 ticks in 10s exactly	**RESOLVED**
R-11	context_switch() argument offset wrong after compiler stack frame	High	Critical	Offsets verified by disassembling sched.o with objdump; push ebp + 48-byte save frame accounted for	**RESOLVED**
R-12	Task stack overflow under heavy IRQ nesting	Low	High	Each preemption ~188 bytes on 8KB stack; max nesting ~42 — well within budget	**MONITORED**
R-13	kmalloc/kfree corruption under concurrent tasks	High	High	CLI/STI guards added to free-list walk in kmalloc() and kfree() during S5 integration	**RESOLVED**
R-14	sched_add_task() enabling interrupts unexpectedly early (STI race)	Med	Med	EFLAGS pushf+cli/push+popf replaces bare cli+sti; IF restored to pre-call state	**RESOLVED**
R-15	VGA scrolling overwriting kernel data at 0xB8000	Low	Med	VGA buffer at 0xB8000; kernel at 0x100000+; no overlap — confirmed by memory map analysis	**MONITORED**

7.3.2 Post-SRS Discovered Risks (R-16 to R-19) — Discovered During S5 Integration

ID	Description	Root Cause	Impact Observed	Resolution
R-16	context_switch .resume: missing ret instruction	Debugging session removed ret, incorrectly reasoning about ESP state after IRET. Only manifested under full integration (not in S4 isolation tests).	Triple fault → infinite QEMU reboot loop. Entire S5 integration blocked.	Restored ret at .resume; added detailed explanatory comment block in context_switch.asm to prevent recurrence. (B-06)
R-17	Heap physical backing region not reserved in bitmap	mm_init() freed all GRUB-available frames above 1MB, including 0x500000–0x8FFFFF which backs the kernel heap. Kernel frames were protected but heap frames were not.	mm_alloc_frame() returned frames overlapping heap. Page tables and TCBs overwrote live heap data → non-deterministic crashes under multi-task load.	Reserve 0x500000–0x8FFFFF in mm_init() after kernel protection loop. mm/mm_phys.c. (B-01)
R-18	registers_t field order inverted vs NASM PUSHAD	PUSHAD pushes EAX first (highest addr), EDI last (lowest addr). Struct declared eax at lowest offset — opposite of stack layout.	Latent bug: future handlers reading regs->eax would silently receive regs->edi. No current crash since only int_no/err_code/eip/cs/eflags were read (above PUSHAD block).	Corrected field order in include/isr.h: edi, esi, ebp, esp_dummy, ebx, edx, ecx, eax. (B-03)

R-19	sched_add_task() bare STI re-enables interrupts unconditionally	CLI+STI pattern assumed interrupts always enabled at call time. If called from CLI context, STI re-enabled IRQs before task was fully enqueued.	Race: IRQ0 could fire before task_count incremented, causing sched_switch() to see task_count < 2 and skip scheduling.	Replaced bare cli+sti with EFLAGS pushf+cli / push+popf save-restore. scheduler/sched.c. (B-04)

7.3.3 Risk Governance Process

The following governance rules were enforced throughout the 40-day development cycle:

- Sprint-End Risk Review: A formal risk review was conducted at the conclusion of each sprint to reassess identified risks, identify newly emerging risks, and update probability/impact classifications.
- Alpha Manager Responsibility: The designated Alpha Manager of the current sprint reported unresolved or escalated risks to the team and ensured mitigation strategies were actively implemented.
- High-Impact Risk Escalation: Risks categorized as High Impact were discussed in a mandatory team review before the next sprint began. No sprint started until mitigation direction for critical risks was agreed upon.
- Mitigation Effectiveness Validation: The effectiveness of all mitigation strategies was formally evaluated during Sprint 5 integration testing and verified against the NFR test suite.
- Risk Documentation: All risk updates, status changes, and resolutions were documented in Project_review_log.md and reflected in the RTM.

7.4 Project Management Tools

The team used a Claude AI-based progress management tool for sprint-based task tracking throughout the project lifecycle. This replaced the traditional Trello board approach with AI-assisted task organization, dependency tracking, and sprint card management.

Column / State	Purpose
Backlog	All tasks not yet started, drawn from SRS requirements with requirement ID tags
In Progress	Tasks currently being developed by an assigned team member
Under Review	Tasks awaiting Alpha Manager review and sign-off
Completed	Fully reviewed, tested, and merged tasks

Each sprint card referenced: the SRS Requirement ID it implements (e.g., REQ-MEM-03), the assigned team member, the expected completion date, and post-implementation test validation notes. This ensured direct traceability between SRS requirements and implementation tasks.

7.5 Coding Standards and Design Template

7.5.1 Function Naming Conventions

Module	Naming Pattern	Concrete Examples
Memory Management	mm_<verb>_<noun>()	mm_alloc_frame(), mm_free_frame(), mm_heap_init(), mm_init()
Paging	paging_<action>()	paging_create_directory(), paging_map_page(), paging_enable(), paging_load_directory(), paging_free_directory()
Scheduler	sched_<action>()	sched_init(), sched_switch(), sched_add_task(), sched_start(), sched_next()
Interrupt Handlers	isr_<vector>()	isr_14() for Page Fault, isr_13() for GPF
PIT / PIC drivers	pit_<action>(), pic_<action>()	pit_init(), pit_get_ticks(), pit_irq0_handler(), pic_remap(), pic_send_eoi()
VGA / Serial	vga_<action>(), serial_<action>()	vga_init(), vga_puts(), vga_set_color(), serial_init(), serial_puts(), serial_printf()

Rules: All function names shall be lowercase with underscores. A module prefix is mandatory. Function names shall be verb-based to convey action.

7.5.2 File Organization Structure

Directory	Contents
boot/	Multiboot header and kernel entry point (boot.asm — NASM: Multiboot magic, stack setup, call kernel_main)
arch/x86/	ALL hardware-specific code: GDT, IDT, PIC, PIT, paging, context switch assembly, ISR stubs — per NFR-PORT-01
mm/	Physical memory bitmap allocator (mm_phys.c) and kernel heap kmalloc/kfree (heap.c)
scheduler/	Round-Robin scheduler (sched.c), TCB definitions and task create/destroy (task.c)

kernel/	kernel_main.c (init sequence), s5_integration.c (NFR test suite), serial.c (COM1 logger), vga.c (text driver)
include/	All header files with #ifndef / #define / #endif header guards

7.5.3 Function Header Comment Template

All non-trivial functions include the following header comment block immediately above the function definition:

```
/*
 * Function: mm_alloc_frame
 * Description: Allocates one 4KB physical frame from the bitmap pool.
 * Implements: REQ-MEM-03
 * Input: None
 * Output: Physical address of allocated frame, or 0 if no frames available
 * Author: Aman Yadav (B24CS1006)
 */
```

Inline comments are mandatory for bitwise operations, register manipulations, and memory boundary logic — the areas most likely to introduce subtle bugs and most difficult to review without context.

7.5.4 Header Guard Template

```
#ifndef MM_PHYS_H
#define MM_PHYS_H
/* type declarations and function prototypes */
#endif /* MM_PHYS_H */
```

7.5.5 SRS Traceability in Code

Every function that implements a numbered SRS requirement includes a traceability comment referencing the requirement ID. This comment appears directly above the function definition:

```
/* Implements: REQ-MEM-03 */
uint32_t mm_alloc_frame(void) { ... }
```

This practice ensures that during code review and integration testing, every implemented function can be traced directly back to a specific requirement in this SRS document.

Appendix A: Glossary

Term / Acronym	Definition
TCB	Task Control Block — data structure storing the complete CPU context and metadata of a process
ISR	Interrupt Service Routine — a handler function invoked by the CPU when an interrupt occurs
MMU	Memory Management Unit — hardware unit responsible for virtual-to-physical address translation
PIT	Programmable Interval Timer — hardware timer used to generate periodic interrupts (IRQ0 at 100 Hz)
PIC	Programmable Interrupt Controller — manages and routes hardware IRQ signals to the CPU (8259A)
GDT	Global Descriptor Table — x86 data structure defining memory segment descriptors
IDT	Interrupt Descriptor Table — x86 data structure mapping interrupt vectors to handler routines
DPL	Descriptor Privilege Level — privilege level encoded in segment/gate descriptors (0=kernel, 3=user)
IRQ	Interrupt Request — a hardware signal requesting CPU attention
CR0 / CR3	Control Registers — CR0 bit 31 (PG) enables paging; CR3 holds the page directory base address
EOI	End of Interrupt — signal sent to PIC (port 0x20) after an IRQ handler completes
ELF	Executable and Linkable Format — binary format used for the kernel image (kernel.elf)
TLB	Translation Lookaside Buffer — CPU cache of recent page table lookups; flushed automatically on CR3 write
IRET	Interrupt Return — x86 instruction that atomically restores EIP, CS, and EFLAGS
PUSHAD	Push All Doubleword — saves EAX, ECX, EDX, EBX, ESP, EBP, ESI, EDI onto the stack (EAX first = highest address)
POPAD	Pop All Doubleword — restores EDI, ESI, EBP, (skip ESP), EBX, EDX, ECX, EAX
TSC	Time Stamp Counter — 64-bit counter incremented every CPU cycle; read by RDTSC instruction

RDTSC	Read Time Stamp Counter — x86 instruction used in NFR-PERF-01 to measure context switch latency

Appendix B: UML Diagram Inventory

The following UML diagrams accompany this SRS as separate deliverables. All diagrams are internally consistent with the requirements defined in Sections 4 and 5 and reflect the final v3.2 implementation.

#	Diagram Type	Purpose	Requirements Covered	Status
1	**Use Case Diagram**	Identifies system actors and high-level use cases for memory and task operations	REQ-MEM-01 to REQ-MEM-05, REQ-TASK-01 to REQ-TASK-05	**DELIVERED** ✓
2	**Class Diagram**	Models key data structures: GDT entry, IDT entry, Page Directory/Table, TCB, Scheduler Queue	REQ-MEM-02, REQ-MEM-03, REQ-TASK-05	**DELIVERED** ✓
3	**Component Diagram**	Shows subsystem boundaries and inter-module dependencies (boot→mm→scheduler→arch/x86)	EXT-SW-01, NFR-MAINT-01	**DELIVERED** ✓
4	**Sequence Diagram (Context Switch)**	Depicts full message sequence: IRQ0 → pit_irq0_handler → sched_switch → context_switch → IRET	REQ-TASK-02, REQ-TASK-03, REQ-TASK-04	**DELIVERED** ✓
5	**Timing Diagram (10ms Quantum)**	Shows temporal behavior of 100Hz scheduler tick across three tasks A→B→C	REQ-TASK-02, NFR-PERF-01, NFR-PERF-02	**DELIVERED** ✓
6	**Activity Diagram**	Models Round-Robin scheduling decision logic: sched_next() flow and TASK_READY state selection	REQ-TASK-03	**DELIVERED** ✓
7	**Composite Structure Diagram**	Shows internal structure of TCB and relationships to kernel stack, page directory, scheduler queue	REQ-TASK-05, REQ-MEM-05	**DELIVERED** ✓
8	**Deployment Diagram**	Shows kernel deployment: ISO image → GRUB → protected mode entry → kernel_main on QEMU i386	Section 2.4 (Operating Environment)	**DELIVERED** ✓

Appendix C: NFR Test Evidence (v3.2 Addition)

All Non-Functional Requirements were formally verified in Sprint 5 via the automated test suite implemented in kernel/s5_integration.c. The test suite runs before the scheduler starts and produces a complete pass/fail report on COM1 serial output.

Test invocation in kernel_main.c (step 17 of 22):

```
__asm__ volatile ("sti");    // IRQs on — PIT must tick for PERF-02 timing
s5_run_all_tests();          // current_task=NULL → sched_switch() is a no-op during
tests
__asm__ volatile ("cli");
```

Test	NFR	Test Function	Method	Measured Result	Verdict
T-01	**EXT-SW-01**	s5_test_ext_sw01()	Create PD, verify zeroed, map user page, free PD	4 sub-tests PASS	**PASS ✓**
T-02	**NFR-PERF-01**	s5_test_perf01_context_switch()	RDTSC over 1000 sched_next() calls; CPUID verifies TSC	~300 cycles avg ≈ 0.3 µs (budget: 50 µs)	**PASS ✓**
T-03	**NFR-PERF-02**	s5_test_perf02_pit_stability()	Count ticks over 10-second wait using pit_get_ticks()	1000 ticks in 10s (±0 ticks)	**PASS ✓**
T-04	**NFR-SEC-01**	s5_test_sec01_kernel_isolation()	Read CR3, walk PD[768–1023], check U/S bit = 0 on all present PTEs	0 violations / 2048+ PTEs	**PASS ✓**
T-05	**NFR-REL-01**	s5_test_sec01_kernel_isolation()	SIDT → read IDT[14] handler address (off_lo + off_hi)	IDT[14] = 0x001035XX ≠ 0	**PASS ✓**
T-06	**Heap Stress**	s5_test_heap_stress()	30 allocs + pattern write/verify + fragment + re-alloc + free all	All 4 phases PASS	**PASS ✓**
T-07	**NFR-MAINT-01**	s5_audit_naming_and_arch()	Static audit of function prefix compliance and header guard presence	All prefixes conform	**PASS ✓**
T-08	**NFR-PORT-01**	s5_audit_naming_and_arch()	Static audit: no CR0/CR3/port I/O outside arch/x86/	No violations detected	**PASS ✓**

Final S5 Score: 14 PASS, 0 FAIL — STATUS: ALL TESTS PASSED

Appendix D: SRS Deviation Log — v3.1.2 → v3.2 (v3.2 Addition)

The following table documents cases where the final implementation deviates from the original SRS v3.1.2 specification. All deviations were approved during Sprint 5 integration review and Sprint 6 documentation.

Item	SRS v3.1.2 Spec	Actual Implementation	Justification
NFR-PERF-01 measurement method	No specific measurement method specified	RDTSC-based TSC cycle count for 1000 sched_next() calls; CPUID verifies TSC availability	PIT resolution (10 ms) is too coarse to measure a single context switch (~300 cycles). TSC provides nanosecond precision and is available on all QEMU x86 targets.
EXT-SW-01 per-task address spaces	Per-task page directories for process isolation	All kernel tasks share single CR3 (kernel page directory); paging_create/free_directory() tested standalone by s5_test_ext_sw01()	Kernel-space tasks do not require separate address spaces. The interface contract is verified; full per-process isolation is out of scope for this educational kernel.
context_switch .resume ret	ret assumed by design at .resume label	ret was accidentally omitted during S4 debugging; restored in S5 integration (R-16, B-06)	Implementation error. The omission caused an infinite QEMU reboot loop discoverable only under full system integration. Restored and documented.
Heap thread safety (S4)	'Raunak adds locking in S4'	CLI/STI interrupt guards added in S5 integration once the scheduler went live (R-13, B-05)	S4 was tested single-threaded in isolation. Guards were added when concurrent heap access became possible after sched_start().
Heap physical reservation	Not specified in SRS v3.1.2	Heap physical frames 0x500000–0x8FFFFF marked allocated in mm_init() after kernel protection (R-17, B-01)	Latent bug: mm_alloc_frame() could return frames overlapping heap backing region. Not covered by original spec. Fixed proactively.
registers_t field layout	Not explicitly specified in SRS	GP register field order corrected to match NASM PUSHAD sequence (EDI lowest, EAX highest) in include/isr.h (R-18, B-03)	Original order was inverted vs actual stack layout — a latent bug for any handler reading regs->eax. Corrected proactively in S5.
sched_add_task interrupt guard	'CLI around queue mutations'	EFLAGS pushf+cli / push+popf replaces bare cli+sti (R-19, B-04)	Bare STI would re-enable interrupts even if caller had them disabled, creating a race window before task_count incremented.

Appendix E: Bug Register (v3.2 Addition)

The following bugs were discovered and resolved during development. All bugs are RESOLVED as of Sprint 6.

ID	Sprint	Title	Root Cause	Symptom	Fix Applied
B-01	S5	**Heap physical frames not reserved in bitmap**	mm_init() freed all GRUB-available frames above 1 MB without excluding heap backing region (0x500000–0x8FFFFF)	Non-deterministic crashes under multi-task load; page tables and TCBs overwriting live heap data	Reserve 0x500000–0x8FFFFF in mm_init() after kernel protection loop. mm/mm_phys.c. (R-17)
B-02	S3	**IDT default entries pointing to address 0 (Present=1)**	Default IDT fill used flags=0x8E (Present=1) with handler base=0x00000000 for entries 48–255	Any stray interrupt on vector 48–255 jumps to address 0 and triple faults — non-recoverable	Changed default fill to flags=0x0E (Present=0) — not-present gate causes controlled #NP exception instead of blind jump to 0. arch/x86/idt.c.
B-03	S5	**registers_t field order inverted vs NASM PUSHAD**	NASM PUSHAD pushes EAX first (highest addr), EDI last (lowest addr); C struct declared eax at lowest offset — opposite of actual stack layout	Latent bug: any handler reading regs->eax receives regs->edi silently (no current handler reads GPRs, so no immediate crash)	Corrected field order in include/isr.h: edi, esi, ebp, esp_dummy, ebx, edx, ecx, eax. (R-18)
B-04	S4/S5	**sched_add_task() unconditional STI creating race condition**	Bare CLI+STI assumed interrupts were enabled at call time; if called from a CLI context, STI unconditionally re-enabled IRQs	Race: IRQ0 fires before task_count is incremented; sched_switch() sees task_count < 2 and skips scheduling for that tick	EFLAGS pushf+cli / push+popf replaces bare cli+sti in sched_add_task(). scheduler/sched.c. (R-19)
B-05	S4/S5	**kmalloc / kfree not interrupt-safe under concurrent tasks**	Free-list walk had no interrupt guard; IRQ0 could preempt mid-walk during a context switch	Double-allocation, next-pointer corruption, free-list cycles under concurrent tasks with live heap access	CLI/STI guards wrap the entire free-list walk in both kmalloc() and kfree(). mm/heap.c. (R-13)
B-06	S5	**context_switch .resume label missing ret instruction**	ret removed from .resume during S4 debugging session that incorrectly reasoned about ESP state after IRET vs before save block. Bug	After IRET to .resume, CPU falls through to garbage bytes → #UD (invalid opcode) → double fault → triple fault → infinite QEMU reset loop.	Restored ret at .resume in arch/x86/context_switch.asm with detailed comment explaining the stack state at that point. (R-16)

			invisible in S4 isolation tests.	Blocked entire S5 integration.	

All 6 bugs RESOLVED. MiniOS v3.2 passes all 14 NFR tests with 0 failures. Submitted April 15, 2026.